CS-462 – Programming Language Foundations
Lecture #13: References and Recursive Types

EPFL

In this lecture, we will develop types for mutable references, in the style of OCaml, as well as recursive types.

1 Reference Types

The syntax and semantics for references is defined as follows.

$$
\begin{aligned}
e &::= \cdots \mid \mathsf{ref}\ e \mid !e \mid e_1 := e_2 \mid \ell \\
v &::= \cdots \mid \ell \\
E &::= \cdots \mid \mathsf{ref}\ E \mid !E \mid E := e \mid v := E
\end{aligned}
$$

$$
\textsc{Alloc}\ \frac{}{\langle \mathsf{ref}\ v, \sigma\rangle \longrightarrow \langle \ell, \sigma[\ell \mapsto v]\rangle}\ \ell \notin dom(\sigma) \qquad \textsc{Deref}\ \frac{}{\langle\ !\ell, \sigma\rangle \longrightarrow \langle v, \sigma\rangle}\ \sigma(\ell) = v
$$

$$
\textsc{Assign}\ \frac{}{\langle \ell := v, \sigma\rangle \longrightarrow \langle (), \sigma[\ell \mapsto v]\rangle}\ \ell \in dom(\sigma)
$$

To extend the type system, we add a new type, τ **ref**, to stand for the type of a location that contains a value of type τ. For example the expression ref 7 has type **int ref**, since it evaluates to a location that contains a value of type **int**. Dereferencing a location of type τ **ref** results in a value of type τ, so $!e$ has type τ if e has type τ **ref**. And for assignment $e_1 := e_2$, if e_1 has type τ **ref**, then e_2 must have type τ. As in OCaml, an assignment is performed only for its effect on the store, so it evaluates to () and has type **unit**.

$$
\tau ::= \cdots \mid \tau\ \mathbf{ref}
$$

$$
\frac{\Gamma \vdash e : \tau}{\Gamma \vdash \mathsf{ref}\ e : \tau\ \mathbf{ref}} \qquad \frac{\Gamma \vdash e : \tau\ \mathbf{ref}}{\Gamma \vdash\ !e : \tau} \qquad \frac{\Gamma \vdash e_1 : \tau\ \mathbf{ref} \quad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 := e_2 : \mathbf{unit}}
$$

Note that there is no typing rule for location values. What should the type of a location value ℓ be? Clearly, it should be of type τ **ref**, where τ is the type of the value contained in location ℓ. But how do we know what value is contained in location ℓ? We could directly examine the store, but this would be inefficient. In addition, examining the store directly may not give us a conclusive answer! Consider, for example, a store σ and location ℓ where $\sigma(\ell) = \ell$; what is the type of ℓ?

Instead, we introduce *store typings* to track the types of values stored in locations. Store typings are partial functions from locations to types. We use metavariable Σ to range over store typings. Our typing relation now becomes a relation over 4 entities: typing contexts, store typings, expressions, and types. We write $\Gamma, \Sigma \vdash e : \tau$ when expression e has type τ under typing context Γ and store typing Σ.

Our new typing rules for references are as follows. (Typing rules for other constructs are modified to take a store typing in the obvious way.)

$$\frac{\Gamma, \Sigma \vdash e : \tau}{\Gamma, \Sigma \vdash \mathsf{ref}\, e : \tau\ \mathbf{ref}} \qquad \frac{\Gamma, \Sigma \vdash e : \tau\ \mathbf{ref}}{\Gamma, \Sigma \vdash\ !e : \tau} \qquad \frac{\Gamma, \Sigma \vdash e_1 : \tau\ \mathbf{ref} \quad \Gamma, \Sigma \vdash e_2 : \tau}{\Gamma, \Sigma \vdash e_1 := e_2 : \mathbf{unit}} \qquad \frac{\Sigma(\ell) = \tau}{\Gamma, \Sigma \vdash \ell : \tau\ \mathbf{ref}}$$

So, how do we state type soundness? Our type soundness theorem for simply-typed lambda calculus said that if $\Gamma \vdash e : \tau$ and $e \longrightarrow^* e'$ then e' is not stuck. But our operational semantics for references now has a store, and our typing judgment now has a store typing in addition to a typing context. We need to adapt the definition of type soundness appropriately. To do so, we define what it means for a store to be well-typed with respect to a typing context.

Definition. Store σ is *well-typed* with respect to typing context Γ and store typing Σ, written $\Gamma, \Sigma \vdash \sigma$, if $dom(\sigma) = dom(\Sigma)$ and for all $\ell \in dom(\sigma)$ we have $\Gamma, \Sigma \vdash \sigma(\ell) : \Sigma(\ell)$.

We can now state type soundness for our language with references.

Theorem (Type soundness)**.** *If* $\cdot, \Sigma \vdash e : \tau$ *and* $\cdot, \Sigma \vdash \sigma$ *and* $\langle e, \sigma \rangle \longrightarrow^* \langle e', \sigma' \rangle$ *then either* e' *is a value, or there exists* e'' *and* σ'' *such that* $\langle e', \sigma' \rangle \longrightarrow \langle e'', \sigma'' \rangle$.

We can prove type soundness for our language using the same strategy as for the simply-typed lambda calculus: using the preservation and progress lemmas. The progress lemma can be easily adapted for the semantics and type system for references. Adapting preservation is a little more involved, since we need to describe how the store typing changes as the store evolves. The rule Alloc extends the store σ with a fresh location ℓ, producing store σ'. Since $dom(\Sigma) = dom(\sigma) \neq dom(\sigma')$, it means that we will not have σ' well-typed with respect to typing store Σ.

Since the store can increase in size during the evaluation of the program, we also need to allow the store typing to grow as well.

Lemma (Preservation)**.** *If* $\Gamma, \Sigma \vdash e : \tau$ *and* $\Gamma, \Sigma \vdash \sigma$ *and* $\langle e, \sigma \rangle \longrightarrow \langle e', \sigma' \rangle$ *then there exists some* $\Sigma' \supseteq \Sigma$ *such that* $\Gamma, \Sigma' \vdash e' : \tau$ *and* $\Gamma, \Sigma' \vdash \sigma'$.

We write $\Sigma' \supseteq \Sigma$ to mean that for all $\ell \in dom(\Sigma)$ we have $\Sigma(\ell) = \Sigma'(\ell)$. This makes sense if we think of partial functions as sets of pairs: $\Sigma \equiv \{(\ell, \tau) \mid \ell \in dom(\Sigma) \wedge \Sigma(\ell) = \tau\}$. Note that the preservation lemma states simply that there is some store type $\Sigma' \supseteq \Sigma$, but does not specify what exactly that store typing is. Intuitively, Σ' will either be Σ, or Σ extended with a newly allocated location.

Interestingly, references are enough to recover Turing completeness. For example, to implement a recursive function f we can initialize a reference cell containing a dummy value for f and then "backpatch" it with the actual definition. For example, here is an implementation of the familiar factorial function, written using let expressions, conditionals, and integers for clarity.

$$\begin{array}{l}
\mathsf{let}\ r = \mathsf{ref}\ \lambda x : \mathbf{int}.\ 0\ \mathsf{in} \\
\mathsf{let}\ f = \lambda x : \mathbf{int}.\ \mathsf{if}\ x = 0\ \mathsf{then}\ 1\ \mathsf{else}\ x \times (\ !r)\ (x - 1)\ \mathsf{in} \\
\mathsf{let}\ a = (r := f)\ \mathsf{in} \\
f\ 5
\end{array}$$

The function f calls whatever function is stored in r, and after the assignment, that is f itself, so $f\ 5$ evaluates to 120. This trick is known as "Landin's knot" after its inventor.

2 Recursive Types

Many programming languages have the ability to define recursive data types. For example, suppose we want to define binary trees with integer data at the nodes. In Java we can write

```
class Tree {
   Tree leftChild, rightChild;
   int data;
}
```

A binary tree is an object of this class. In OCaml we can write

```
type tree = Leaf | Node of int * tree * tree
```

These types are *recursive* because they are defined in terms of themselves.

In the simply-typed lambda calculus, we do not yet have any mechanism to define recursive types. We would like the type **tree** to satisfy

$$\mathit{tree} = \mathbf{unit} + \mathbf{int} \times \mathit{tree} \times \mathit{tree},$$

In other words, we would like **tree** to be a solution of the equation

$$\alpha = \mathbf{unit} + \mathbf{int} \times \alpha \times \alpha$$

However, no such solution exists among the types we have seen so far.

How might we augment our set of types to include solutions to such recursive type equations? There are two basic approaches, called the *equirecursive* and *isorecursive* approach, respectively.

3 Equirecursive Types

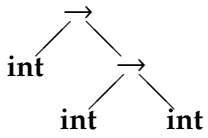
By unwinding the equation above, we can see that

$$\begin{aligned}
\alpha &= \mathbf{unit} + \mathbf{int} \times \alpha \times \alpha \\
&= \mathbf{unit} + \mathbf{int} \times (\mathbf{unit} + \mathbf{int} \times \alpha \times \alpha) \times (\mathbf{unit} + \mathbf{int} \times \alpha \times \alpha) \\
&= \mathbf{unit} + \mathbf{int} \times (\mathbf{unit} + \mathbf{int} \times (\mathbf{unit} + \mathbf{int} \times \alpha \times \alpha) \times (\mathbf{unit} + \mathbf{int} \times \alpha \times \alpha)) \times \\
&\qquad\qquad (\mathbf{unit} + \mathbf{int} \times (\mathbf{unit} + \mathbf{int} \times \alpha \times \alpha) \times (\mathbf{unit} + \mathbf{int} \times \alpha \times \alpha)) \\
&= \cdots
\end{aligned}$$

At each level, we have a finite type with the type variable α appearing at some of the leaves, and we obtain the next level by substituting the right-hand side of the equation for α. This gives a sequence of deeper and deeper finite trees, where each successive tree is a substitution instance of the previous tree.

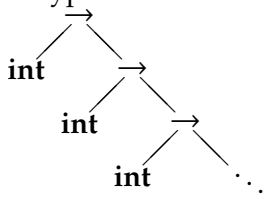
If we take the limit of this process, we have an infinite tree. We can think of this as an infinite labeled graph whose nodes are labeled with the type constructors $\times$, $+$, **int**, and **unit**. This is very much like an ordinary type expression, except that it is infinite. There are no more α's, because we have substituted for all of them all the way down. This infinite tree is a solution of our equation, and this is what we take as the type **tree**.

More generally, over standard type constructors such as $\rightarrow$, $\times$, $+$, **unit**, and **int**, we can form the set of (finite) types inductively in the usual way. Each such type can be regarded as a finite labeled tree. For example, the type $\mathbf{int} \rightarrow \mathbf{int} \rightarrow \mathbf{int}$ can be viewed as the labeled tree



Now let us add some infinite types. These are infinite labeled trees that respect the arities of the constructors; that is, if the constructor is binary (such as $\times$ or $\rightarrow$), any node labeled with that constructor must have exactly two children; and if the constructor is nullary, such as **unit**, then any node labeled with that symbol must be a leaf. Within these constraints, the tree may be infinite.

A (finite or infinite) expression with only finitely many subexpressions (up to isomorphism) is called *regular*. For example, the infinite type



is regular, since it has only two subexpressions up to isomorphism, namely itself and **int**. The limit of the unwinding of the equation above, which we took to be the type **tree**, is also regular; it has exactly five subexpressions up to isomorphism, namely **tree**, **unit**, $\mathbf{int} \times \mathbf{tree} \times \mathbf{tree}$, $\mathbf{tree} \times \mathbf{tree}$, and **int**.

Regular trees are all we need to provide solutions to finite systems of type equations. Suppose we have n type equations in n variables:

$$\begin{aligned} \alpha_1 &= \tau_1 \\ &\vdots \\ \alpha_n &= \tau_n, \end{aligned} \tag{1}$$

where each τ_i is a finite type over the type constructors and type variables $\alpha_1 \ldots \alpha_n$. This system has a solution $\sigma_1 \ldots \sigma_n$ in which each σ_i is a regular tree. Moreover, if no right-hand side is a variable, then the solution is unique.

3.1 The μ Constructor

We can specify the infinite solutions to systems of type equations using a finite syntax involving a new type constructor μ, the *fixed-point type constructor*. If we have an equation $\alpha = \tau$ such that the right-hand side is not α, then there is a unique solution, which is a finite or infinite regular tree. The solution will be infinite if α occurs in τ and will be finite (in fact it will just be τ) if α does not occur in τ. We denote this unique solution by $\mu\alpha.\,\tau$.

Syntactically, μ acts as a binding operator in type expressions as λ does in λ-terms, with the same notions of scope, free and bound variables, α-conversion, and substitution.

Since $\mu\alpha.\,\tau$ is a solution to $\alpha = \tau$, we have

$$\mu\alpha.\,\tau = \tau\{\mu\alpha.\,\tau/\alpha\}.$$

For example, to get a **tree** type satisfying our original equation, we can define

$$\mathbf{tree} \triangleq \mu\alpha.\,\mathbf{unit} + \mathbf{int} \times \alpha \times \alpha.$$

The solutions $\sigma_1 \ldots \sigma_n$ to any finite system of equations can be expressed in terms of μ. For example, suppose τ_1 and τ_2 are finite type expressions over the type variables α_1, α_2 such that neither τ_1 nor τ_2 is a variable. The system

$$\alpha_1 = \tau_1 \qquad\qquad \alpha_2 = \tau_2$$

has a unique solution σ_1, σ_2 specified by

$$\sigma_1 = \mu\alpha_1.(\tau_1\{\mu\alpha_2.\,\tau_2/\alpha_2\}) \qquad\qquad \sigma_2 = \mu\alpha_2.(\tau_2\{\mu\alpha_1.\,\tau_1/\alpha_1\}).$$

Mutually recursive type declarations arise quite often in practice. For example, consider the following Java class definitions for `Node` and `Edge`:

```
class Node {
   Edge[] inEdges, outEdges;
}
class Edge {
   Node source, sink;
}
```

Note that `Node` refers to `Edge` and vice versa. So we must take a mutual fixed point.

3.2 Typing Rules

In the equirecursive view, since $\mu\alpha.\,\tau = \tau\{\mu\alpha.\,\tau/\alpha\}$, the typing rules are simple:

$$\mu\text{-INTRO}\ \frac{\Gamma \vdash e : \tau\{\mu\alpha.\,\tau/\alpha\}}{\Gamma \vdash e : \mu\alpha.\,\tau} \qquad\qquad \mu\text{-ELIM}\ \frac{\Gamma \vdash e : \mu\alpha.\,\tau}{\Gamma \vdash e : \tau\{\mu\alpha.\,\tau/\alpha\}}$$

Equivalently, we can just allow substitution of equals for equals in type expressions.

4 Isorecursive Types

There is another approach to recursive types, the *isorecursive* approach. Here we do not have any infinite types, but rather the expression $\mu\alpha.\,\tau$ is itself a type. In this approach, $\mu\alpha.\,\tau$ and $\tau\{\mu\alpha.\,\tau/\alpha\}$ are considered distinct (but isomorphic) types.

The step of substituting $\mu\alpha.\,\tau$ for α in τ is called *unfolding*, and the reverse operation is called *folding*. The conversion of elements between these two types is accomplished by explicit fold and unfold operations.

$$\begin{aligned} \mathsf{unfold}_{\mu\alpha.\,\tau} &: \mu\alpha.\,\tau \rightarrow \tau\{\mu\alpha.\,\tau/\alpha\} \\ \mathsf{fold}_{\mu\alpha.\,\tau} &: \tau\{\mu\alpha.\,\tau/\alpha\} \rightarrow \mu\alpha.\,\tau \end{aligned}$$

We will often suppress the subscripts when there is no ambiguity. In this view, the equality symbol in a recursive equation is not really an equality, but an isomorphism.

4.1 Typing Rules

In the isorecursive view, the typing rules consist of a pair of introduction and elimination rules for μ-types that explicitly mention fold and unfold:

$$\mu\text{-INTRO}\ \frac{\Gamma \vdash e : \tau\{\mu\alpha.\,\tau/\alpha\}}{\Gamma \vdash \mathsf{fold}\ e : \mu\alpha.\,\tau} \qquad\qquad \mu\text{-ELIM}\ \frac{\Gamma \vdash e : \mu\alpha.\,\tau}{\Gamma \vdash \mathsf{unfold}\ e : \tau\{\mu\alpha.\,\tau/\alpha\}}$$

4.2 Operational Semantics

With isorecursive types, we also need to augment the operational semantics. Under CBV, fold v is a value, and we evaluate the arguments of fold and unfold first:

$$v ::= \cdots \mid \mathsf{fold}\ v$$
$$E ::= \cdots \mid \mathsf{fold}\ E \mid \mathsf{unfold}\ E$$

Then we only need one new rule:

$$\frac{}{\mathsf{unfold}\ (\mathsf{fold}\ v) \longrightarrow v}$$

Intuitively, to access data in a recursive type $\mu\alpha.\,\tau$, we need to unfold it first; but the only way that values of type $\mu\alpha.\,\tau$ could have been created in the first place is via a fold.

4.3 An Example

Suppose we want to write a program to add a list of numbers. The list type is a recursive type, which we can define as $\textbf{intlist} \triangleq \mu\alpha.\,\textbf{unit} + \textbf{int} \times \alpha$. Now suppose we want to add up the elements of an **intlist**. This will be a recursive function, so we would need to take a fixed point. In the body of this function, we would like to do a case on the **intlist** l. But to do a case, we need a sum type, and l is a μ-type, so we will have to unfold it first. (OCaml does this automatically when it sees a match.) So the body would be

$$\begin{array}{l}\mathsf{case}\ \mathsf{unfold}\ l\ \mathsf{of}\\ \quad (\lambda u : \textbf{unit}.\,0)\\ \mid (\lambda p : \textbf{int} \times \textbf{intlist}.\,(\#1\ p) + f\ (\#2\ p))\end{array}$$

This is just the same code that you would write in OCaml, except we have broken out some of the things that OCaml hides for you. In particular, we have explicitly shown the recursion in the definition of the **intlist** type and the unfold that is needed to get the exploded view of the type.

5 Equirecursive vs. Isorecursive

Programming languages deal with recursive types in different ways. Java and Modula-3 take the equirecursive approach, in which the folded and unfolded types are considered equal, and the fold/unfold operations are just the identity functions. Recursive types and their unfoldings are fully substitutable for each other.

```
class E {
   String x;
   E e;
   public String toString() {
      return e.e.e.e.e.e.e.e.e.e.e.e.e.e.e.x;
   }
}
```

On the other hand, the ML family, CLU, and C use isorecursive types, in which $\mu\alpha.\,\tau$ and $\tau\{\mu\alpha.\,\tau/\alpha\}$ are considered different (but isomorphic) types, and the casting operators fold and unfold are required to go between them. CLU uses "up" and "down" instead of fold and unfold. In OCaml, the unfold operator is performed automatically and implicitly by the "match" and "let" statements and the pattern-matching mechanism. The type constructors in a recursive datatype definition, applied to arguments, act as fold operations.

```
# type tree = Leaf | Node of int * tree * tree;;
type tree = Leaf | Node of int * tree * tree
# Node (4, Leaf, Leaf);;
- : tree = Node (4, Leaf, Leaf)
```

6 Numbers as a Recursive Type

We started with primitive types **unit**, **bool**, and **int**. But the type **bool** can be represented as **unit** + **unit**, with the values true and false represented by the left and right injections, respectively.

Now that we have recursive types, we no longer need to take **int** as primitive, but we can define it as a recursive type. A natural number is either 0 or a successor of a natural number. Thus we can take

$$\begin{aligned}\mathbf{nat} &\triangleq \mu\alpha.\,\mathbf{unit} + \alpha\\ 0 &\triangleq \mathsf{fold}\ (\mathsf{inl}_{\mathbf{unit+nat}}\ ())\\ 1 &\triangleq \mathsf{fold}\ (\mathsf{inr}_{\mathbf{unit+nat}}\ 0)\\ 2 &\triangleq \mathsf{fold}\ (\mathsf{inr}_{\mathbf{unit+nat}}\ 1),\end{aligned}$$

and so on. We can use the recursive type **nat** to code up all of the usual arithmetic, and all these operations are well-typed. For example, the successor function would be

$$(\lambda x : \mathbf{nat}.\,\mathsf{fold}\ (\mathsf{inr}_{\mathbf{unit+nat}}\ x)) : \mathbf{nat} \to \mathbf{nat}.$$

So all we really need as primitive types and type constructors are **unit**, functions, products, sums, and recursive types. With these we can build all the other types like natural numbers, integers, lists, trees, floating point numbers, and so on.

7 Self-Application and Ω

Recall the diverging term Ω defined by

$$\omega \triangleq \lambda x.\,x\ x \qquad\qquad \Omega \triangleq \omega\ \omega.$$

We can now give these terms recursive types, provided we insert some folding and unfolding. Since x is applied as a function, it must have some kind of function type, say $\sigma \to \tau$. But since it is applied to itself as an argument, it must also have type σ. This seems to indicate that the type of x must satisfy the equation $\sigma = \sigma \to \tau$. The recursive type $\mu\alpha.\,(\alpha \to \tau)$ appears to be in order (here τ can be anything).

To actually apply x to x, we have to unfold it. The type of $\mathsf{unfold}\ x$ is

$$\mathsf{unfold}\ x : (\mu\alpha.\,(\alpha \to \tau)) \to \tau.$$

This is a function with domain $\mu\alpha.\,(\alpha \to \tau)$, which is the type of x, so we can apply it to x. The type of the result $(\mathsf{unfold}\ x)\ x$ is τ. Thus the fully typed ω term is

$$\omega \triangleq (\lambda x : \mu\alpha.\,(\alpha \to \tau).\,(\mathsf{unfold}\ x)\ x)\ :\ (\mu\alpha.\,(\alpha \to \tau)) \to \tau.$$

If we now fold this, we get

$$\mathsf{fold}\ \omega : \mu\alpha.\,(\alpha \to \tau).$$

Therefore, we can apply ω as a function to $\mathsf{fold}\ \omega$, and the result is

$$\omega\ (\mathsf{fold}\ \omega) : \tau.$$

This is the same as the original Ω term, but with explicit folding and unfolding.

We can do this in OCaml:

```
# type u = Fold of (u -> u);;
type u = Fold of (u -> u)
# let omega = fun x -> match x with Fold f -> f x;;
val omega : u -> u = <fun>
# omega (Fold omega);;
...runs forever until you hit control-c
```

So recursive types alone, without fix or references, let us write programs that run forever. The point is that this program passed type checking: it is well-typed.

8 Untyped to Typed λ-Calculus

In fact, with recursive types, we can type everything in the pure untyped lambda calculus. Every λ-term can be applied as a function to any other λ-term, so every λ-term (with appropriate folds and unfolds inserted) has type

$$U \triangleq \mu\alpha.\,\alpha \to \alpha$$

The translation is

$$\begin{aligned}
[\![x]\!] &\triangleq x \\
[\![e_0\ e_1]\!] &\triangleq (\mathsf{unfold}\ [\![e_0]\!])\ [\![e_1]\!] \\
[\![\lambda x.\,e]\!] &\triangleq \mathsf{fold}\ \lambda x : U.\,[\![e]\!].
\end{aligned}$$

Note that every untyped term maps to a term of type U.